

Windows Privilege Escalation: Juicy Potato

Your `SeImpersonatePrivilege` guide's Part 2 covered the general "Potato" pattern conceptually — this guide covers the tool that started that entire naming convention and made the pattern famous: Juicy Potato, along with its actively-maintained modern successor JuicyPotatoNG, since the original tool's specific trigger mechanism has been patched on current Windows versions. Worth locking in the distinction immediately: **Part 1 covers exactly how Juicy Potato's original COM-based trigger mechanism works, including why it stopped working on patched systems — Part 2 covers practical usage, CLSID selection, and JuicyPotatoNG's updated approach**, since successfully running this tool requires target-specific configuration choices the original tool's mechanical explanation alone doesn't cover.

1. The Specific Trigger Mechanism — The Foundation

Your `SeImpersonatePrivilege` guide's Section 5 described the general Potato pattern abstractly: stand up a listener, trigger a SYSTEM process into authenticating to it, impersonate the resulting token. Juicy Potato's specific implementation of that "trigger" step abuses **DCOM (Distributed Component Object Model)** activation combined with a quirk in how certain COM servers authenticate over NTLM:

1. Juicy Potato instructs Windows to activate a SPECIFIC COM object (identified by its CLSID - Class ID) that is known to run under a SYSTEM-privileged context (specifically, the BITS - Background Intelligent Transfer Service - COM server was the classic, original target)
2. Juicy Potato ALSO stands up a fake OXID resolver (a component of DCOM's own object-location/activation infrastructure) on a LOCAL port it controls
3. When Windows attempts to activate the requested SYSTEM-context COM object, the resulting NTLM authentication handshake gets REDIRECTED to Juicy Potato's fake OXID resolver instead of the genuine one, because Juicy Potato has manipulated the activation request to point there
4. That authentication handshake carries the SYSTEM-privileged COM server's TOKEN with it - Juicy Potato captures and impersonates this token, exactly per the general pattern your `SeImpersonatePrivilege` guide described

The single fact that explains why this specific tool eventually stopped working on modern systems — Microsoft's patches specifically closed the OXID resolver redirection quirk (Section 1's step 3) that the original technique depended on, meaning the underlying `SelmpersonatePrivilege` precondition remains completely valid and exploitable today, but THIS SPECIFIC mechanical trigger for obtaining a `SYSTEM` token no longer functions on fully patched, modern Windows versions — directly the "patch arms race" pattern your `SelmpersonatePrivilege` guide's Section 6 described, made concrete with this tool as the first, most famous example.

PART 1: Understanding CLSID Selection & Version Limitations

2. Why CLSID Choice Matters

The COM object Juicy Potato instructs Windows to activate (Section 1, step 1) is specified by its CLSID — and not every CLSID works identically across every Windows version. A CLSID valid and exploitable on Windows Server 2016 may not be present, may run under a different privilege context, or may not trigger the redirection quirk at all on Windows 10 or Windows Server 2019/2022:

```
Juicy Potato requires specifying:  
-c <CLSID> → the specific COM object to trigger activation of  
  
Community-maintained CLSID lists (searchable specifically for  
"juicy potato CLSID list windows [version]") catalog which CLSIDs  
are known-working per target Windows version - checking the  
target's EXACT OS version/build before selecting a CLSID is a  
required step, not an optional refinement
```

3. Version Applicability — Where Juicy Potato (Original) Still Works

```
Original Juicy Potato: reliably works on Windows Server 2016 and  
EARLIER, and on Windows 10 builds prior to the specific update  
that patched the OXID resolver redirection quirk (Section 1)  
  
Windows Server 2019, Windows Server 2022, and current-patch-level  
Windows 10/11: the original technique's core trigger mechanism  
is patched and NON-FUNCTIONAL - this is precisely why  
JuicyPotatoNG (Part 2) exists as a maintained, updated alternative
```

Confirming the target's exact OS version and patch level (`systeminfo` , or `[Environment]::OSVersion` via PowerShell) before attempting this specific tool is a required first step — running the original Juicy Potato against a modern, patched target will simply fail, and that failure alone doesn't indicate the broader SelmpersonatePrivilege-based approach is unviable, only that THIS specific variant's trigger mechanism is patched.

PART 2: Practical Usage & the JuicyPotatoNG Update

4. Running Original Juicy Potato

```
JuicyPotato.exe -l 1337 -p C:\Windows\System32\cmd.exe -t *  
-c {CLSID-HERE}
```

<code>-l 1337</code>	→ local port for the fake OXID resolver to listen on
<code>-p PATH</code>	→ the process to launch once a SYSTEM token is successfully captured and impersonated (a command shell here, but any executable works)
<code>-t *</code>	→ impersonation/creation type - '*' tries both <code>CreateProcessWithToken</code> and <code>CreateProcessAsUser</code> methods, letting the tool pick whichever succeeds given the current account's exact privilege set
<code>-c {CLSID-HERE}</code>	→ the specific COM object CLSID selected per Section 2's version-appropriate list

5. JuicyPotatoNG — The Maintained, Modern Successor

Developed specifically to restore functionality against Windows versions where the original tool's Section 1 mechanism was patched, using an updated set of techniques/CLSIDs and additional refinements to the core approach:

```
JuicyPotatoNG.exe -t * -p "C:\Windows\System32\cmd.exe" -a
```

<code>-a</code>	→ automatically test/select a working CLSID from JuicyPotatoNG's own updated internal list, rather than requiring manual CLSID research/selection the way the original tool does - a genuinely significant usability improvement given how version-sensitive Section 2's manual CLSID selection process otherwise is
-----------------	--

JuicyPotatoNG maintains compatibility further into modern Windows

Server 2019/2022 and current Windows 10/11 builds specifically because its underlying technique incorporates updates addressing the exact patches that broke the original tool

6. Confirming Successful Exploitation

```
# JuicyPotatoNG/Juicy Potato launches the specified process  
# (Section 4's -p argument) directly with the captured SYSTEM  
# token - once launched, confirm within THAT new process:  
whoami  
# expected output: nt authority\system
```

7. Troubleshooting Common Failure Modes

"CreateProcessWithTokenW failed" or similar errors → often indicates SeImpersonatePrivilege isn't actually enabled in the current context (recheck via whoami /priv per your SeImpersonatePrivilege guide's Section 2), or that the specific CLSID selected doesn't apply to this exact target version

No output / hangs indefinitely → often indicates a firewall or local security policy is blocking the local listener port (-l argument) - try a different port, or confirm no local firewall rule is interfering

Works but launched process isn't actually SYSTEM → often indicates the WRONG CLSID was selected for this target's specific version (Section 2) - the activation succeeded, but not against a genuinely SYSTEM-context COM server

8. Testing Methodology

1. Confirm SeImpersonatePrivilege per your SeImpersonatePrivilege guide's Section 2, as the baseline precondition
2. Determine exact target OS version/build (systeminfo) to assess which variant (original vs JuicyPotatoNG) is likely to work (Section 3)
3. For modern targets (Server 2019+/current Windows 10-11), start with JuicyPotatoNG's -a auto-selection (Section 5) rather than attempting manual CLSID research against the original tool first
4. For older targets, select an appropriate CLSID from a version-matched community list (Section 2) and run the original tool
5. If the first attempt fails, work through Section 7's troubleshooting checklist before concluding the technique is unviable - many failures are configuration/CLSID-select

ion issues rather than a fundamentally blocked precondition

6. If Juicy Potato/JuicyPotatoNG both fail against a specific target, proceed to the PrintSpoofer guide as an alternative trigger mechanism relying on the SAME underlying SeImpersonatePrivilege precondition

9. Legal & Ethical Note

Identical baseline authorization requirement to every guide in this series - standard for any privilege-escalation tooling covered throughout this and the companion Linux series.

10. Prevention

- Patch Windows systems to current levels - directly closes the ORIGINAL Juicy Potato's specific trigger mechanism (Section 1), though notably does NOT eliminate the underlying SeImpersonatePrivilege-based technique family entirely, given JuicyPotatoNG and PrintSpoofer's continued relevance against patched systems via different trigger mechanisms
- Apply least privilege to service account SeImpersonatePrivilege assignment, per your SeImpersonatePrivilege guide's Section 9 - this remains the single most direct upstream fix for the ENTIRE Potato exploit family, since every specific tool (this one, PrintSpoofer, and others) shares the identical underlying precondition regardless of which specific trigger mechanism each uses
- Monitor for anomalous local DCOM activation requests and unexpected local OXID resolver connections as a detection signature specific to this tool family

11. Practical Lab Example

Directly continues your SeImpersonatePrivilege guide's own lab setup - a service-account foothold with the privilege confirmed present.

Suggested practice order:

1. On an OLDER lab VM (Windows Server 2016 or early Windows 10 build, matched to Section 3's applicability window), run original Juicy Potato with a version-appropriate CLSID and confirm SYSTEM access
2. On a MODERN, fully patched lab VM, confirm the original tool fails, then run JuicyPotatoNG's -a auto-selection against the SAME target and confirm it succeeds where the original didn't -

- directly observing Section 1's patch-arms-race point in practice
3. Proceed into the PrintSpoofer guide's lab exercise against the SAME modern VM as a second, independent confirmation path

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management) - same umbrella CWE as the SeImpersonatePrivilege and Token Impersonation guides this technique implements
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	Critical (identical severity profile to every technique in this exploit family, given the direct SYSTEM-level outcome)
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1134.001 (Access Token Manipulation: Token Impersonation/Theft) - same technique ID as the Token Impersonation and SeImpersonatePrivilege guides, since this tool is one specific implementation of that broader technique

Quick Reference Cheat Sheet

ORIGINAL JUICY POTATO (older targets only - Server 2016 and earlier, older Windows 10 builds):
JuicyPotato.exe -l 1337 -p C:\Windows\System32\cmd.exe -t * -c {CLSID}
→ requires a VERSION-MATCHED CLSID from a community list

JUICYPOTATONG (modern targets - Server 2019/2022, current Windows 10/11):
JuicyPotatoNG.exe -t * -p "C:\Windows\System32\cmd.exe" -a
→ auto-selects a working CLSID, no manual research needed

CONFIRM:
whoami (run FROM the newly-spawned process) → expect
nt authority\system

TROUBLESHOOT:
CreateProcessWithTokenW failed → recheck SeImpersonatePrivilege
Hangs/no output → check local firewall on listener port
Wrong resulting identity → wrong CLSID for this target version

Root lesson: this tool's own history IS the lesson - a technique

that worked reliably for years, got patched, and required an actively-maintained successor (JuicyPotatoNG) to keep working against modern targets, while the underlying PRIVILEGE precondition never changed at all. Expect this pattern to keep repeating; it's exactly why the PrintSpoofer guide next covers a mechanically DIFFERENT trigger against the SAME precondition, as a genuinely independent fallback rather than just "another Potato variant."

Good next study step: move to the PrintSpoofer guide next — it abuses a completely different Windows component (the Print Spooler service, rather than DCOM/COM activation) to reach the identical SYSTEM-token outcome, making it a genuinely independent technique to fall back on when Juicy Potato/JuicyPotatoNG both fail against a specific hardened or unusually-configured target.